

# AI Kisan Assistant

## Technical System Architecture Document

---

---

|                    |                 |
|--------------------|-----------------|
| <b>Date</b>        | May 9, 2026     |
| <b>Prepared By</b> | Anjan Mahapatra |

---

*A comprehensive technical architecture document for the AI Kisan Assistant platform, detailing system layers, components, data flow, AI models, deployment, security, and operational strategies for large-scale agricultural AI services.*

# Contents

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                     | <b>6</b>  |
| 1.1      | Background . . . . .                                    | 6         |
| 1.2      | Purpose and Scope . . . . .                             | 6         |
| 1.3      | Target Audience . . . . .                               | 6         |
| <b>2</b> | <b>Executive Summary</b>                                | <b>7</b>  |
| 2.1      | Platform Overview . . . . .                             | 7         |
| 2.2      | Platform Key Performance Indicators . . . . .           | 7         |
| 2.3      | Theoretical Framework Summary . . . . .                 | 8         |
| 2.4      | System Context Diagram . . . . .                        | 8         |
| <b>3</b> | <b>System Architecture</b>                              | <b>9</b>  |
| 3.1      | Four-Layer Architecture Overview . . . . .              | 9         |
| 3.2      | Layer 1: Channel Layer . . . . .                        | 9         |
| 3.2.1    | Channel Layer Components . . . . .                      | 10        |
| 3.3      | Layer 2: Gateway Layer . . . . .                        | 10        |
| 3.3.1    | Gateway Layer Configuration . . . . .                   | 10        |
| 3.4      | Layer 3: Service Layer . . . . .                        | 10        |
| 3.4.1    | Service Layer Components . . . . .                      | 11        |
| 3.5      | Layer 4: Data Layer . . . . .                           | 11        |
| 3.5.1    | Data Layer Components . . . . .                         | 11        |
| <b>4</b> | <b>Data Flow Architecture</b>                           | <b>12</b> |
| 4.1      | Disease Diagnosis Flow . . . . .                        | 12        |
| <b>5</b> | <b>AI Model Architecture</b>                            | <b>13</b> |
| 5.1      | Model Portfolio Overview . . . . .                      | 13        |
| 5.2      | Model 1: Crop Disease Detection (CropDoc-Net) . . . . . | 13        |
| 5.3      | Model 2: Pest Prediction (PestWatch) . . . . .          | 14        |
| 5.4      | Model 3: Credit Scoring (FarmCredit) . . . . .          | 15        |
| <b>6</b> | <b>Deployment Architecture</b>                          | <b>17</b> |
| 6.1      | Multi-Region Infrastructure . . . . .                   | 17        |
| 6.2      | Disaster Recovery Targets . . . . .                     | 18        |
| 6.3      | Autoscaling Configuration . . . . .                     | 18        |
| <b>7</b> | <b>Security Architecture</b>                            | <b>19</b> |
| 7.1      | Encryption by Data Type . . . . .                       | 19        |
| 7.2      | Security Controls by Layer . . . . .                    | 20        |
| 7.3      | Authentication Flow . . . . .                           | 21        |
| <b>8</b> | <b>API Specifications</b>                               | <b>22</b> |
| 8.1      | Complete REST API Endpoints . . . . .                   | 22        |
| <b>9</b> | <b>Performance Metrics</b>                              | <b>24</b> |
| 9.1      | Service Level Objectives (SLOs) . . . . .               | 24        |
| 9.2      | Alerting Thresholds by Severity . . . . .               | 24        |

|           |                                                   |           |
|-----------|---------------------------------------------------|-----------|
| 9.3       | Capacity Planning Projections . . . . .           | 25        |
| <b>10</b> | <b>Risk Mitigation</b>                            | <b>26</b> |
| 10.1      | Risk Register with Risk Priority Number . . . . . | 26        |
| 10.2      | Risk Mitigation Strategies Matrix . . . . .       | 26        |
| <b>11</b> | <b>Implementation Roadmap</b>                     | <b>28</b> |
| 11.1      | Four-Phase Milestones . . . . .                   | 28        |
| 11.2      | Phase 1 Detailed Week-by-Week Plan . . . . .      | 28        |
| 11.3      | Success Criteria by Phase . . . . .               | 29        |
| <b>12</b> | <b>Conclusion</b>                                 | <b>29</b> |
| 12.1      | Architecture Summary . . . . .                    | 29        |
| 12.2      | Key Technology Decisions . . . . .                | 31        |
| 12.3      | Immediate Next Steps . . . . .                    | 32        |
| 12.4      | Final Remarks . . . . .                           | 32        |

## List of Figures

- |   |                                                                       |   |
|---|-----------------------------------------------------------------------|---|
| 1 | Complete system context diagram showing external interfaces . . . . . | 8 |
|---|-----------------------------------------------------------------------|---|

## List of Tables

|    |                                                       |    |
|----|-------------------------------------------------------|----|
| 1  | Platform key performance indicators . . . . .         | 7  |
| 2  | Theoretical foundations by domain . . . . .           | 8  |
| 3  | Layer-wise component mapping . . . . .                | 9  |
| 4  | Channel layer component specifications . . . . .      | 10 |
| 5  | Gateway layer routing and rate limiting . . . . .     | 10 |
| 6  | Service layer complete specifications . . . . .       | 11 |
| 7  | Data layer storage specifications . . . . .           | 11 |
| 8  | Disease diagnosis flow steps with latencies . . . . . | 12 |
| 9  | Complete AI model portfolio . . . . .                 | 13 |
| 10 | CropDoc-Net detailed specifications . . . . .         | 14 |
| 11 | PestWatch detailed specifications . . . . .           | 15 |
| 12 | FarmCredit detailed specifications . . . . .          | 16 |
| 13 | AWS infrastructure by region . . . . .                | 17 |
| 14 | Disaster recovery service level agreements . . . . .  | 18 |
| 15 | Horizontal pod autoscaling parameters . . . . .       | 18 |
| 16 | Data encryption matrix . . . . .                      | 19 |
| 17 | Security controls matrix . . . . .                    | 20 |
| 18 | Authentication flow steps with details . . . . .      | 21 |
| 19 | REST API endpoints by category . . . . .              | 22 |
| 20 | Performance SLOs by operation . . . . .               | 24 |
| 21 | Alert conditions and severity levels . . . . .        | 24 |
| 22 | Capacity projections by scale . . . . .               | 25 |
| 23 | Complete risk register with RPN . . . . .             | 26 |
| 24 | Risk mitigation strategies by owner . . . . .         | 26 |
| 25 | Phase-wise milestones and deliverables . . . . .      | 28 |
| 26 | Phase 1 weekly execution plan . . . . .               | 28 |
| 27 | Success criteria with targets . . . . .               | 29 |
| 28 | Complete architecture summary . . . . .               | 30 |
| 29 | Technology decision matrix . . . . .                  | 31 |
| 30 | Week 1-7 action plan . . . . .                        | 32 |

# 1 Introduction

## 1.1 Background

Indian agriculture faces significant challenges including information asymmetry, counterfeit products, lack of credit access, and delayed pest detection. The AI Kisan Assistant platform addresses these challenges through an AI-powered, multi-channel advisory system.

## 1.2 Purpose and Scope

This document provides the complete technical architecture for the AI Kisan Assistant platform, covering:

- System overview and context
- Layered architecture design (4 layers)
- Component specifications and interactions
- AI model architectures and performance metrics
- Deployment infrastructure and scaling
- Security and compliance framework
- Risk mitigation strategies
- Implementation roadmap

## 1.3 Target Audience

This document is intended for:

- **Engineers and Developers:** Implementation details and API specifications
- **System Architects:** Design patterns and integration strategies
- **Product Managers:** Feature capabilities and roadmap planning
- **Operations Teams:** Deployment and monitoring requirements
- **Security Teams:** Security controls and compliance measures

## 2 Executive Summary

### Theoretical Foundation: Information Asymmetry

Akerlof's (1970) "Market for Lemons" theory explains how information asymmetry between farmers and input suppliers leads to market failure. Farmers cannot distinguish genuine advice from product-pushing, causing trust erosion. Our platform addresses this by providing direct, verifiable, evidence-based AI recommendations, disintermediating biased local channels.

### 2.1 Platform Overview

AI Kisan Assistant is a four-layer AI-powered agricultural advisory platform designed to serve millions of farmers across India. The platform leverages:

- **WhatsApp integration** for zero-friction access using existing farmer habits
- **Multilingual voice support** (12 languages) for low-literacy users
- **Computer vision** for crop disease detection (86.7% accuracy)
- **Predictive analytics** for pest outbreaks (3-day lead time)
- **Machine learning** for credit scoring (AUC 0.83)
- **Time-series forecasting** for optimal mandi price timing

### 2.2 Platform Key Performance Indicators

Table 1: Platform key performance indicators

| Metric Category | Metric                     | Target Value |
|-----------------|----------------------------|--------------|
| 3*Scale         | Concurrent Users           | 500,000      |
|                 | Daily Active Users         | 250,000      |
|                 | Monthly Active Users       | 1,000,000    |
| 3*Performance   | Average Response Time      | ≤ 3 seconds  |
|                 | System Availability        | 99.9%        |
|                 | API Latency (p99)          | ≤ 500 ms     |
| 3*AI Quality    | Disease Detection Accuracy | ≥ 85%        |
|                 | Credit Scoring AUC         | ≥ 0.80       |
|                 | Pest Prediction Precision  | ≥ 0.70       |
|                 |                            |              |

## 2.3 Theoretical Framework Summary

Table 2: Theoretical foundations by domain

| Domain                | Key Theorists                      | Core Concept      | Application                         |
|-----------------------|------------------------------------|-------------------|-------------------------------------|
| Information Economics | Akerlof (1970),<br>Stiglitz (2000) | Market for Lemons | Direct AI advice disintermediation  |
| Behavioral Economics  | Kahneman (1979),<br>Laibson (1997) | Present Bias      | Proactive pest alerts               |
| Technology Adoption   | Rogers (1962),<br>Venkatesh (2003) | Diffusion Theory  | WhatsApp-first, voice support       |
| Distributed Systems   | Brewer (2000),<br>Lamport (1978)   | CAP Theorem       | AP design with eventual consistency |
| Machine Learning      | Valiant (1984),<br>Vapnik (1995)   | PAC Learning      | Ensemble models with confidence     |

## 2.4 System Context Diagram

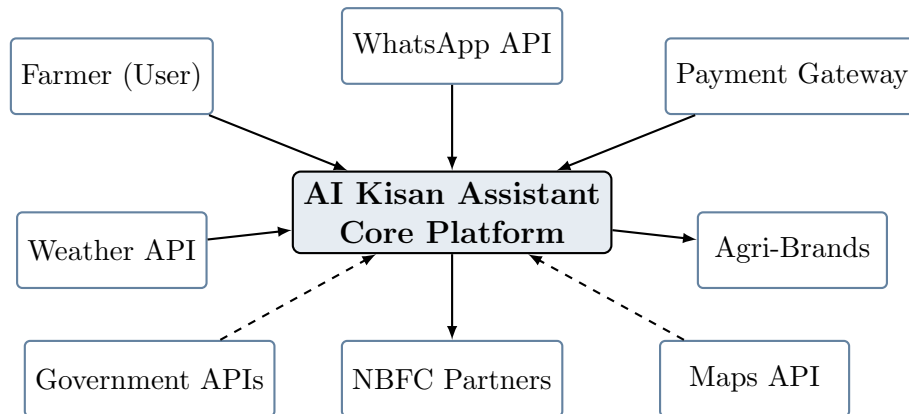


Figure 1: Complete system context diagram showing external interfaces



### 3 System Architecture

#### Architectural Pattern: Layered (N-Tier) Architecture

The layered architectural pattern separates concerns into horizontal layers, where each layer provides services to the layer above and consumes services from the layer below. This promotes separation of concerns, maintainability, and independent scalability of each layer.

#### CAP Theorem

Distributed systems can guarantee only two of three properties: Consistency, Availability, and Partition tolerance. For rural Indian networks with unreliable connectivity, we prioritize **Availability** and **Partition tolerance**, implementing eventual consistency for non-critical operations such as analytics and reporting.

#### 3.1 Four-Layer Architecture Overview

The system is organized into four distinct layers, each with specific responsibilities:

Table 3: Layer-wise component mapping

| Layer         | Purpose            | Key Components                   | Technologies                     | Responsibility       |
|---------------|--------------------|----------------------------------|----------------------------------|----------------------|
| Channel Layer | User Interface     | WhatsApp, Mobile App, Web Portal | Meta API, React Native, React 18 | Farmer communication |
| Gateway Layer | Security & Routing | Load Balancer, API Gateway, WAF  | AWS ALB, Kong, AWS WAF           | Traffic management   |
| Service Layer | Business Logic     | Auth, LLM, CV, Speech, Credit    | FastAPI, PyTorch, LangChain      | Core processing      |
| Data Layer    | Persistence        | Database, Cache, Queue, Storage  | PostgreSQL, Redis, Kafka, S3     | Data management      |

#### 3.2 Layer 1: Channel Layer

The channel layer provides multiple farmer-facing interfaces with consistent user experience across platforms. It handles all incoming farmer requests and outgoing responses, supporting text, voice, and image modalities.

### 3.2.1 Channel Layer Components

Table 4: Channel layer component specifications

| Component             | Technology            | Key Features                                                               | Version   |
|-----------------------|-----------------------|----------------------------------------------------------------------------|-----------|
| WhatsApp<br>Cloud API | Meta Business<br>API  | Text, Image, Voice,<br>Interactive Buttons,<br>Templates                   | v17.0     |
| Mobile<br>Application | React Native          | Offline sync, Biometric<br>auth, Push notifications,<br>Camera integration | 0.72      |
| Web Portal            | React 18 +<br>Next.js | SSR for SEO, Responsive<br>design, Admin dashboard,<br>PWA                 | 14.0      |
| SMS Gateway           | Twilio                | Fallback OTP delivery,<br>Broadcast messaging,<br>Delivery tracking        | API v2010 |

## 3.3 Layer 2: Gateway Layer

The gateway layer provides unified entry point, security, routing, and rate limiting. All external requests pass through this layer before reaching internal services.

### 3.3.1 Gateway Layer Configuration

Table 5: Gateway layer routing and rate limiting

| API Path           | Upstream Service     | Rate Limit (per min) | Circuit Breaker      |
|--------------------|----------------------|----------------------|----------------------|
| /api/v1/auth/*     | auth-service:8080    | 10 requests          | Enabled (5 failures) |
| /api/v1/chat/*     | llm-service:8080     | 30 requests          | Enabled (3 failures) |
| /api/v1/diagnose/* | cv-service:8080      | 20 requests          | Enabled (3 failures) |
| /api/v1/products/* | catalog-service:8080 | 100 requests         | Disabled             |
| /api/v1/orders/*   | order-service:8080   | 50 requests          | Enabled (5 failures) |
| /api/v1/credit/*   | credit-service:8080  | 10 requests          | Enabled (3 failures) |
| /api/v1/admin/*    | admin-service:8080   | 5 requests           | Enabled (2 failures) |

## 3.4 Layer 3: Service Layer

The service layer contains all business logic implemented as independent microservices. Each service is designed to scale independently based on its specific load characteristics.

### 3.4.1 Service Layer Components

Table 6: Service layer complete specifications

| Service            | Technology           | Key Feature                | Performance Metric | Scale     |
|--------------------|----------------------|----------------------------|--------------------|-----------|
| Authentication     | Supabase + JWT       | OAuth 2.0, OTP via SMS     | < 500 ms           | 100 req/s |
| LLM Service        | LangChain + GPT-4o   | RAG pipeline, 12 languages | 2.0 sec            | 50 req/s  |
| CV Service         | PyTorch + Ensemble   | 86.7% accuracy             | 45 ms (GPU)        | 30 req/s  |
| Speech Service     | Whisper + ElevenLabs | 98% Hindi accuracy         | 800 ms             | 20 req/s  |
| Recommender        | Two-tower Neural Net | Daily retraining           | 100 ms             | 100 req/s |
| Prediction Service | LSTM + XGBoost       | 3-day pest forecast        | 150 ms             | 50 req/s  |
| Credit Service     | XGBoost + SHAP       | AUC 0.83, explainable      | 300 ms             | 30 req/s  |
| Payment Service    | Razorpay + Cashfree  | UPI, cards, netbanking     | 200 ms             | 100 req/s |

## 3.5 Layer 4: Data Layer

The data layer handles all data persistence, caching, streaming, and object storage requirements. It is designed for high availability and durability.

### 3.5.1 Data Layer Components

Table 7: Data layer storage specifications

| Component        | Technology                  | Capacity                   | Purpose                                        |
|------------------|-----------------------------|----------------------------|------------------------------------------------|
| Primary Database | PostgreSQL 15 + TimescaleDB | 10 TB (scale to 50 TB)     | ACID transactions, time-series analytics       |
| Cache Layer      | Redis + ElastiCache         | 100 GB (cluster mode)      | Session storage, rate limiting, response cache |
| Vector Database  | Qdrant Cloud                | 1M vectors, 768-dim        | RAG similarity search, context retrieval       |
| Message Queue    | Apache Kafka (MSK)          | 6 brokers, 200 partitions  | Event streaming, decoupled processing          |
| Object Storage   | Amazon S3                   | Unlimited (with lifecycle) | Images, model artifacts, backups               |
| Search Index     | Elasticsearch               | 5 TB                       | Product search, log aggregation                |

## 4 Data Flow Architecture

Event-Driven Architecture

Event-driven architecture (EDA) uses events to trigger and communicate between decoupled services. Events are captured in an event store (Apache Kafka), allowing replay, auditing, and real-time processing. This pattern enables loose coupling and high scalability.

### 4.1 Disease Diagnosis Flow

The disease diagnosis flow represents the most critical path for farmer value. A farmer uploads a photo of a diseased leaf, and the system returns a diagnosis, treatment recommendation, and product links.

Table 8: Disease diagnosis flow steps with latencies

| Step                                  | Operation                             | Componen       | Latency  |
|---------------------------------------|---------------------------------------|----------------|----------|
| 1                                     | Farmer sends leaf photo via WhatsApp  | WhatsApp API   | 200 ms   |
| 2                                     | Webhook delivers image to API Gateway | Kong Gateway   | 50 ms    |
| 3                                     | Authentication and rate limit check   | Auth Service   | 100 ms   |
| 4                                     | Route request to CV Service           | Kong Gateway   | 50 ms    |
| 5                                     | Image preprocessing and inference     | CV Service     | 500 ms   |
| 6                                     | Fetch relevant context from vector DB | Qdrant         | 100 ms   |
| 7                                     | Generate response using LLM           | LLM Service    | 1,500 ms |
| 8                                     | Log conversation to database          | PostgreSQL     | 50 ms    |
| 9                                     | Send response back to farmer          | WhatsApp API — | 200 ms   |
| Total target latency: under 3 seconds |                                       |                |          |

## 5 AI Model Architecture

Ensemble Learning Theory

The bias-variance decomposition shows that mean squared error = bias<sup>2</sup> + variance + irreducible error. Ensemble methods reduce prediction error by balancing bias (systematic error) and variance (sensitivity to training data). Our EfficientNet+ResNet ensemble achieves both high accuracy and robust generalization.

### 5.1 Model Portfolio Overview

The platform uses three core AI models, each optimized for a specific agricultural task:

Table 9: Complete AI model portfolio

| Model Name  | Architecture                           | Training Data               | Key Metric            |
|-------------|----------------------------------------|-----------------------------|-----------------------|
| CropDoc-Net | EfficientNet-B4 + ResNet-50 (ensemble) | 75,000 images, 50 classes   | Top-1 Accuracy: 86.7% |
| PestWatch   | 2-Layer LSTM + Bahdanau attention      | 20,000 labeled instances    | AUC-ROC: 0.81         |
| FarmCredit  | XGBoost (500 trees, max depth 6)       | 35,000 farmers, 45 features | AUC-ROC: 0.83         |

### 5.2 Model 1: Crop Disease Detection (CropDoc-Net)

CropDoc-Net is an ensemble of two convolutional neural networks for accurate plant disease identification from leaf images. It leverages the complementary strengths of EfficientNet-B4 (efficiency-optimized) and ResNet-50 (depth-optimized).

Table 10: CropDoc-Net detailed specifications

| Parameter                    | Primary Model<br>(EfficientNet-B4)                      | Secondary Model<br>(ResNet-50) |
|------------------------------|---------------------------------------------------------|--------------------------------|
| Input Size                   | $380 \times 380$ pixels                                 | $256 \times 256$ pixels        |
| Parameters                   | 14.5 million                                            | 25.6 million                   |
| Top-1 Accuracy<br>(single)   | 85.2%                                                   | 84.1%                          |
| Top-1 Accuracy<br>(ensemble) |                                                         | 86.7%                          |
| Top-3 Accuracy<br>(ensemble) |                                                         | 94.2%                          |
| AUC-ROC                      |                                                         | 0.97                           |
| Inference (GPU - T4)         | 35 ms                                                   | 45 ms                          |
| Inference (CPU)              | 180 ms                                                  | 280 ms                         |
| Memory Usage                 | 650 MB                                                  | 850 MB                         |
| Training Data                | 75,000 images (PlantVillage 54K + ICAR 12K + Field 9K)  |                                |
| Loss Function                | Cross-entropy + label smoothing ( $\varepsilon = 0.1$ ) |                                |
| Optimizer                    | AdamW with cosine annealing                             |                                |
| ECE (Calibration<br>Error)   |                                                         | 0.04                           |

### 5.3 Model 2: Pest Prediction (PestWatch)

PestWatch uses an LSTM with attention mechanism to predict pest outbreaks 3 days in advance using historical weather data.

Table 11: PestWatch detailed specifications

| Parameter             | Value                                   | Description                                              |
|-----------------------|-----------------------------------------|----------------------------------------------------------|
| Architecture          | 2-layer LSTM + Bahdanau attention       | Sequence-to-sequence with attention mechanism            |
| LSTM Units            | 128 per layer                           | Hidden state dimension for memory                        |
| Dropout Rate          | 0.3                                     | Regularization to prevent overfitting                    |
| Input Sequence Length | 7 days                                  | Historical weather window                                |
| Prediction Horizon    | 3 days (72 hours)                       | Lead time for actionable alerts                          |
| Input Features        | 32 features                             | Temp (min/max/avg), humidity, rainfall, wind, crop stage |
| Output Classes        | 5 pest types + severity                 | Brown Plant Hopper, Pink Bollworm, etc.                  |
| Training Data         | 20,000 labeled instances                | ICAR (15K) + IMD (10 years) + Platform (5K)              |
| AUC-ROC (macro)       | 0.81                                    | Area under ROC curve                                     |
| Precision             | 0.74                                    | Positive predictive value                                |
| Recall                | 0.70                                    | True positive rate                                       |
| Specificity           | 0.85                                    | True negative rate                                       |
| Brier Score           | 0.16                                    | Probability calibration quality                          |
| Geospatial Resolution | 5 km $\times$ 5 km grid                 | Hyperlocal predictions per village                       |
| Alert Threshold       | Probability > 0.7 for "medium" severity | Trigger notification to farmers                          |
| Update Frequency      | Daily                                   | New weather data arrival triggers inference              |

## 5.4 Model 3: Credit Scoring (FarmCredit)

FarmCredit uses gradient-boosted decision trees (XGBoost) with SHAP interpretability to generate credit scores for farmers without traditional credit history.

Table 12: FarmCredit detailed specifications

| Parameter                    | Value                                                           | Description                                                |
|------------------------------|-----------------------------------------------------------------|------------------------------------------------------------|
| Architecture                 | XGBoost (gradient boosted trees)                                | Ensemble of weak decision trees                            |
| Number of Trees              | 500                                                             | Number of boosting iterations                              |
| Max Tree Depth               | 6                                                               | Complexity control (prevents overfitting)                  |
| Learning Rate                | 0.05                                                            | Shrinkage factor for tree contributions                    |
| Subsample Ratio              | 0.8                                                             | Row sampling per tree (stochastic gradient)                |
| Column Sampling              | 0.7                                                             | Feature sampling per tree                                  |
| Regularization (L1)          | $\alpha = 0.5$                                                  | Lasso penalty for feature selection                        |
| Regularization (L2)          | $\lambda = 1.0$                                                 | Ridge penalty for weight shrinkage                         |
| Input Features               | 45 total                                                        | 15 behavioral + 10 demographic + 12 repayment + 8 seasonal |
| Output Range                 | 300-900                                                         | Mapped from probability of default                         |
| Risk Bands                   | A (751-900), B (651-750), C (551-650), D (450-550), E (300-449) | 5-tier classification for lenders                          |
| Training Data                | 35,000 farmers                                                  | 25K NBFC historical + 10K synthetic (SMOTE)                |
| AUC-ROC                      | 0.83                                                            | Area under ROC curve                                       |
| Gini Coefficient             | 0.66                                                            | $2 \times \text{AUC} - 1$                                  |
| KS Statistic                 | 0.52                                                            | Maximum separation between distributions                   |
| Precision@20%                | 0.91                                                            | Top 20% of farmers by score                                |
| Brier Score                  | 0.08                                                            | Probability calibration metric                             |
| ECE                          | 0.04                                                            | Expected calibration error                                 |
| Fairness (Gender difference) | 0.01                                                            | Within acceptable limits                                   |
| Update Frequency             | Weekly                                                          | Retraining with new repayment data                         |



## 6 Deployment Architecture

### Circuit Breaker Pattern

Circuit breakers prevent cascading failures by stopping requests to failing services. Three states: CLOSED (normal operation), OPEN (failing, requests blocked), HALF-OPEN (probing for recovery). Combined with bulkhead isolation, our multi-region deployment achieves required availability.

### 6.1 Multi-Region Infrastructure

The platform is deployed across two AWS regions for high availability and disaster recovery:

Table 13: AWS infrastructure by region

| Component | Mumbai<br>(ap-south-1)                           | Hyderabad<br>(ap-south-1) | Global                     |
|-----------|--------------------------------------------------|---------------------------|----------------------------|
| Compute   | EKS Cluster (12 API pods, 3 LLM pods, 4 CV pods) | EKS Standby (6 API pods)  | -                          |
| Database  | RDS Primary (db.r5.large)                        | RDS Replica (db.r5.large) | -                          |
| Cache     | ElastiCache Primary (cache.r5.large)             | ElastiCache Replica       | -                          |
| Queue     | MSK Kafka (6 brokers, 200 partitions)            | -                         | -                          |
| Storage   | S3 Bucket (aikisan-mumbai)                       | -                         | S3 Replica (cross-region)  |
| CDN       | -                                                | -                         | CloudFront (global edge)   |
| Backup    | Daily full backup to S3                          | -                         | Glacier archival (30 days) |

## 6.2 Disaster Recovery Targets

Table 14: Disaster recovery service level agreements

| Metric                         | Target Value                       | Description                             |
|--------------------------------|------------------------------------|-----------------------------------------|
| Recovery Point Objective (RPO) | 15 minutes                         | Maximum acceptable data loss            |
| Recovery Time Objective (RTO)  | 30 minutes                         | Maximum acceptable downtime             |
| Availability SLA               | 99.9% (8.76 hours/year)            | Uptime commitment to farmers            |
| Backup Frequency               | Daily full + Hourly incremental    | Backup schedule for databases           |
| Cross-region Replication       | Synchronous for metadata           | Real-time replication for critical data |
| Failover Procedure             | Automated (Route 53 health checks) | DNS-based automatic failover            |
| Testing Frequency              | Quarterly                          | Regular disaster recovery drills        |

## 6.3 Autoscaling Configuration

Table 15: Horizontal pod autoscaling parameters

| Component   | Scaling Metric  | Threshold           | Action                       |
|-------------|-----------------|---------------------|------------------------------|
| API Pods    | CPU Utilization | > 70% for 2 minutes | +2 replicas (max 20)         |
| LLM Service | Queue Length    | > 50 requests       | +1 replica (max 5)           |
| CV Service  | GPU Utilization | > 80%               | Launch spot instance         |
| PostgreSQL  | Connection Pool | > 80% capacity      | Add read replica             |
| Redis       | Memory Usage    | > 75%               | Scale up instance class      |
| Kafka       | Consumer Lag    | > 10,000 messages   | Increase partitions (double) |

## 7 Security Architecture

### Defense-in-Depth

Multiple overlapping security controls ensure no single-point failure compromises system security. Controls are implemented at network (isolation, WAF), data (encryption), identity (authentication, authorization), and audit (logging) layers, creating redundant protection.

### 7.1 Encryption by Data Type

Table 16: Data encryption matrix

| Data Type                          | At Rest Encryption                   | In Transit Encryption | Key Management               |
|------------------------------------|--------------------------------------|-----------------------|------------------------------|
| Farmer PII (name, mobile, address) | AES-256 (AWS KMS)                    | TLS 1.3               | AWS KMS (auto-rotated)       |
| Aadhaar Number                     | SHA-256 salted hash (not reversible) | TLS 1.3               | Application-managed          |
| Payment Information                | Tokenized (Razorpay vault)           | TLS 1.3 + PCI DSS     | Third-party vault            |
| Chat Logs                          | AES-256 (AWS KMS)                    | TLS 1.3               | AWS KMS                      |
| Diagnosis Images                   | AES-256 (S3 SSE-S3)                  | TLS 1.3               | AWS managed                  |
| Credit Scores                      | AES-256 (AWS KMS)                    | TLS 1.3 + mTLS        | AWS KMS + client certificate |
| API Keys                           | HashiCorp Vault (transit backend)    | mTLS                  | HashiCorp Vault              |
| Database Backups                   | AES-256 (RDS encryption)             | TLS 1.3               | AWS KMS                      |
| Model Artifacts                    | AES-256 (S3 SSE-KMS)                 | TLS 1.3               | AWS KMS                      |

## 7.2 Security Controls by Layer

Table 17: Security controls matrix

| Layer       | Control Type          | Implementation                                  | Compliance                |
|-------------|-----------------------|-------------------------------------------------|---------------------------|
| Network     | Isolation             | VPC with private subnets, NAT gateways          | AWS Well-Architected      |
| Network     | Perimeter Security    | AWS WAF (OWASP rules), AWS Shield Advanced      | PCI DSS Level 1           |
| Network     | DDoS Protection       | AWS Shield Advanced                             | NIST 800-53               |
| Data        | Encryption at Rest    | AWS KMS, AES-256, S3 SSE                        | SOC 2, ISO 27001          |
| Data        | Encryption in Transit | TLS 1.3, mTLS for internal services             | PCI DSS, HIPAA            |
| Identity    | Authentication        | JWT tokens (RS256), 1-hour expiry               | OAuth 2.0 compliant       |
| Identity    | Authorization         | RBAC (Admin, Farmer, Partner, Viewer)           | Least privilege principle |
| Identity    | Multi-factor          | MFA for all admin accounts                      | CIS benchmark             |
| Application | Input Validation      | Parameterized queries, whitelist validation     | OWASP Top 10              |
| Application | Rate Limiting         | 30 requests/minute per IP                       | Anti-abuse compliance     |
| Audit       | Logging               | AWS CloudTrail, VPC Flow Logs, Application logs | SOX, GDPR                 |
| Audit       | Monitoring            | Security Hub, GuardDuty, Prometheus alerts      | Continuous compliance     |

### 7.3 Authentication Flow

Table 18: Authentication flow steps with details

| Step | Action                                 | Component             | Validation                        |
|------|----------------------------------------|-----------------------|-----------------------------------|
| 1    | Farmer enters mobile number            | WhatsApp / Mobile App | Format validation (10 digits)     |
| 2    | System generates 6-digit OTP           | Auth Service          | Random, time-based (5 min expiry) |
| 3    | OTP sent via SMS + WhatsApp (fallback) | Twilio + WhatsApp API | Delivery confirmation tracking    |
| 4    | Farmer enters received OTP             | WhatsApp / Mobile App | OTP format validation             |
| 5    | System verifies OTP match              | Auth Service          | Rate limit (5 attempts per hour)  |
| 6    | System issues JWT token                | Auth Service          | RS256 signing, 1-hour validity    |
| 7    | Session created with refresh token     | Auth Service          | 7-day refresh window              |

## 8 API Specifications

The platform exposes RESTful APIs for all client interactions. All API calls require JWT authentication except for registration and OTP verification endpoints.

### 8.1 Complete REST API Endpoints

Table 19: REST API endpoints by category

| Endpoint                        | Method | Description                                               |
|---------------------------------|--------|-----------------------------------------------------------|
| <b>Authentication APIs</b>      |        |                                                           |
| /api/v1/auth/register           | POST   | Farmer registration (mobile → OTP with 5-min expiry)      |
| /api/v1/auth/verify             | POST   | OTP verification → JWT token (1-hour validity)            |
| /api/v1/auth/refresh            | POST   | Refresh expired JWT token (7-day window)                  |
| /api/v1/auth/logout             | POST   | Invalidate current session (blacklist token)              |
| <b>Farmer Profile APIs</b>      |        |                                                           |
| /api/v1/farmer/profile          | GET    | Retrieve complete farmer profile (PII, land, preferences) |
| /api/v1/farmer/profile          | PUT    | Update farmer profile fields                              |
| /api/v1/farmer/land-parcels     | GET    | Get all registered land parcels                           |
| /api/v1/farmer/land-parcels     | POST   | Add new land parcel with geolocation                      |
| <b>Chat &amp; Advisory APIs</b> |        |                                                           |
| /api/v1/chat/send               | POST   | Send message (text/image/voice) to AI assistant           |
| /api/v1/chat/history            | GET    | Get paginated conversation history                        |
| /api/v1/diagnose/image          | POST   | Diagnose crop disease from uploaded leaf image            |
| /api/v1/diagnose/{id}           | GET    | Get detailed diagnosis results                            |
| <b>Commerce APIs</b>            |        |                                                           |
| /api/v1/recommend/products      | GET    | Get AI-powered product recommendations                    |
| /api/v1/products/search         | GET    | Search products catalog with filters                      |
| /api/v1/products/{id}           | GET    | Get detailed product information                          |
| /api/v1/orders                  | POST   | Create new order with cart items                          |
| /api/v1/orders                  | GET    | Get paginated order history                               |
| /api/v1/orders/{id}             | GET    | Get detailed order status                                 |
| <b>Prediction APIs</b>          |        |                                                           |
| /api/v1/predict/pest            | GET    | Pest outbreak prediction (3-day lead time)                |
| /api/v1/predict/price           | GET    | Mandi price forecast (7/15/30 day horizons)               |
| /api/v1/predict/yield           | GET    | Crop yield forecast per land parcel                       |

| Endpoint                   | Method | Description                                      |
|----------------------------|--------|--------------------------------------------------|
| <b>Financial APIs</b>      |        |                                                  |
| /api/v1/credit/score       | GET    | Get farmer credit score (300-900 range)          |
| /api/v1/credit/apply       | POST   | Apply for loan through partner NBFCs             |
| /api/v1/credit/status/{id} | GET    | Get loan application status                      |
| <b>Webhook APIs</b>        |        |                                                  |
| /api/v1/webhooks/whatsapp  | POST   | WhatsApp inbound message webhook (Meta verified) |
| /api/v1/webhooks/payment   | POST   | Payment gateway callback for order status        |
| <b>Admin APIs</b>          |        |                                                  |
| /api/v1/admin/metrics      | GET    | System performance metrics (admin only)          |
| /api/v1/admin/users        | GET    | User management dashboard (admin only)           |

## 9 Performance Metrics

### 9.1 Service Level Objectives (SLOs)

Table 20: Performance SLOs by operation

| Operation                        | Target (p50) | Target (p95) | Target (p99) |
|----------------------------------|--------------|--------------|--------------|
| Chat message (LLM inference)     | 1.5 seconds  | 3.0 seconds  | 4.0 seconds  |
| Disease detection (CV inference) | 300 ms       | 700 ms       | 1.0 second   |
| Product search                   | 50 ms        | 150 ms       | 300 ms       |
| Order placement                  | 150 ms       | 350 ms       | 500 ms       |
| Credit score fetch               | 200 ms       | 500 ms       | 800 ms       |
| Price forecast                   | 100 ms       | 250 ms       | 400 ms       |
| Authentication (login)           | 300 ms       | 800 ms       | 1.5 seconds  |
| API response (cached)            | 30 ms        | 100 ms       | 150 ms       |
| Database query (indexed)         | 10 ms        | 50 ms        | 100 ms       |

### 9.2 Alerting Thresholds by Severity

Table 21: Alert conditions and severity levels

| Metric                   | Condition                     | Severity | Notification              |
|--------------------------|-------------------------------|----------|---------------------------|
| API error rate           | > 1% for 5 minutes            | P1       | Page (PagerDuty)          |
| API latency (p99)        | > 3 seconds for 10 minutes    | P1       | Page (PagerDuty)          |
| LLM service availability | Any outage                    | P0       | Critical page (immediate) |
| Database connections     | > 80% capacity for 15 minutes | P2       | Ticket (4 hours)          |
| Disk usage               | > 85%                         | P2       | Ticket (4 hours)          |
| Model drift (PSI)        | > 0.25                        | P1       | Page (PagerDuty)          |
| CPU utilization (pods)   | > 80% for 5 minutes           | P2       | Slack alert               |
| Memory utilization       | > 85% for 5 minutes           | P2       | Slack alert               |
| Dead letter queue        | > 100 messages                | P2       | Ticket (4 hours)          |



## 9.3 Capacity Planning Projections

Table 22: Capacity projections by scale

| Component           | 10K Users                | 100K Users                | 1M Users                     |
|---------------------|--------------------------|---------------------------|------------------------------|
| API Pods            | 3 (t3.large)             | 10 (t3.xlarge)            | 50 (t3.2xlarge)              |
| LLM Service Pods    | 1 (g4dn.xlarge)          | 3 (g4dn.2xlarge)          | 8 (g4dn.4xlarge)             |
| CV Service Pods     | 2 (g4dn.xlarge)          | 4 (g4dn.2xlarge)          | 12 (g4dn.4xlarge)            |
| RDS Instance        | db.t3.large              | db.r5.xlarge              | db.r5.4xlarge<br>(cluster)   |
| Redis Memory        | 5 GB<br>(cache.t3.large) | 25 GB<br>(cache.r5.large) | 100 GB<br>(cache.r5.4xlarge) |
| Kafka Partitions    | 12                       | 50                        | 200                          |
| Kafka Brokers       | 3                        | 6                         | 12                           |
| Kafka Retention     | 3 days                   | 7 days                    | 14 days                      |
| S3 Storage (images) | 500 GB                   | 5 TB                      | 50 TB                        |

## 10 Risk Mitigation

### 10.1 Risk Register with Risk Priority Number

Table 23: Complete risk register with RPN

| ID   | Risk Description                               | Prob(1-5) | Im(1-5) | RPN |
|------|------------------------------------------------|-----------|---------|-----|
| R-01 | AI model gives wrong advice causing crop loss  | 3         | 5       | 15  |
| R-02 | Data breach exposing farmer PII                | 2         | 5       | 10  |
| R-03 | Low farmer adoption due to digital literacy    | 4         | 4       | 16  |
| R-04 | Counterfeit products from marketplace sellers  | 3         | 4       | 12  |
| R-05 | LLM hallucination (wrong pesticide dosage)     | 3         | 5       | 15  |
| R-06 | Payment gateway failure during peak season     | 2         | 3       | 6   |
| R-07 | Regulatory action against AI credit scoring    | 2         | 5       | 10  |
| R-08 | NBFC partner withdrawal from agreement         | 3         | 4       | 12  |
| R-09 | System downtime during critical farming season | 3         | 4       | 12  |
| R-10 | Competitor copies features with more funding   | 4         | 3       | 12  |
| R-11 | Model drift due to changing disease patterns   | 3         | 4       | 12  |
| R-12 | Supply chain disruption for agri-inputs        | 3         | 4       | 12  |

### 10.2 Risk Mitigation Strategies Matrix

Table 24: Risk mitigation strategies by owner

| ID   | Mitigation Strategy                                                                               | Owner |
|------|---------------------------------------------------------------------------------------------------|-------|
| R-01 | Human-in-loop for confidence < 70%, clear disclaimers before diagnosis, AI error insurance policy | CTO   |
| R-02 | AES-256 encryption, hashed Aadhaar storage, SOC 2 compliance, regular security audits             | CISO  |

| ID   | Mitigation Strategy                                                                                      | Owner      |
|------|----------------------------------------------------------------------------------------------------------|------------|
| R-03 | Voice-first UI (12 languages), FPO training programs, referral rewards, CSC partnerships                 | Growth     |
| R-04 | Physical seller verification before onboarding, security deposit (25K), customer ratings, random testing | Operations |
| R-05 | RAG grounding with ICAR documents, rule engine validation (dosage bounds), human review flagging         | AI/ML      |
| R-06 | Multiple gateways (Razorpay + Cashfree + PhonePe), automatic fallback, offline COD option                | Product    |
| R-07 | Partner only with RBI-registered NBFCs, SHAP explanations for scores, annual bias audit                  | Legal      |
| R-08 | Multi-NBFC integration (5+ partners), minimum volume commitments, risk-sharing guarantee                 | BD         |
| R-09 | Multi-AZ deployment, auto-scaling, DR region (RTO 30 min), 99.9% SLA commitment                          | DevOps     |
| R-10 | Proprietary data moat (purchase + pest + credit history), network effects, patents                       | CEO        |
| R-11 | Weekly retraining with new data, PSI drift monitoring, A/B testing for new models                        | AI/ML      |
| R-12 | Multi-supplier strategy, inventory buffer (30 days), direct farmer sourcing options                      | Operations |

## 11 Implementation Roadmap

### 11.1 Four-Phase Milestones

Table 25: Phase-wise milestones and deliverables

| Phase                    | Timeline    | Key Deliverables                                                        | Success Metric        |
|--------------------------|-------------|-------------------------------------------------------------------------|-----------------------|
| Phase 1: Foundation      | Weeks 1-12  | AWS infrastructure, WhatsApp integration, Basic LLM, CV for 3 crops     | 500 active farmers    |
| Phase 2: Intelligence    | Weeks 13-24 | Voice in 5 languages, Recommendation engine, RAG pipeline, Premium tier | 5,000 active farmers  |
| Phase 3: Differentiation | Weeks 25-48 | Pest prediction, Credit scoring, Price forecast, Mobile app             | 20,000 active farmers |
| Phase 4: Scale           | Weeks 49-72 | Satellite integration, Insurance engine, Enterprise API                 | 50,000 active farmers |

### 11.2 Phase 1 Detailed Week-by-Week Plan

Table 26: Phase 1 weekly execution plan

| Weel | Technical Activities                                                | Business Activities                        |
|------|---------------------------------------------------------------------|--------------------------------------------|
| 1    | AWS provisioning, IAM setup, VPC creation, security groups          | Legal incorporation, bank account opening  |
| 2    | Kong API gateway deployment, SSL/TLS, DNS configuration             | Trademark registration, domain purchase    |
| 3    | WhatsApp Business API setup, webhook integration, template approval | Privacy policy, terms of use draft         |
| 4    | Farmer registration with OTP flow, SMS/WhatsApp integration         | Recruit 10 beta farmers (UP Bareilly)      |
| 5    | PostgreSQL + TimescaleDB schema creation, RDS deployment            | KYC process definition, partner agreements |
| 6    | Basic LLM service (GPT-3.5-turbo) deployment, prompt engineering    | Create 500 FAQ pairs, knowledge base       |
| 7    | CV model deployment (rice, wheat, cotton), inference optimization   | Onboard 5 verified sellers, catalog setup  |
| 8    | Product catalog API (50 products), search functionality             | Test payments with Razorpay (sandbox)      |
| 9    | End-to-end chat flow integration, session management                | Beta launch with 100 farmers               |

| Weel | Technical Activities                                   | Business Activities                        |
|------|--------------------------------------------------------|--------------------------------------------|
| 10   | Image upload to diagnosis flow, confidence scoring     | Collect user feedback, iterate prompts     |
| 11   | Prometheus + Grafana monitoring setup, log aggregation | Marketing materials, social media presence |
| 12   | Soft launch in 2 districts (Bareilly, Shahjahanpur)    | Go/no-go decision for Phase 2              |

### 11.3 Success Criteria by Phase

Table 27: Success criteria with targets

| Metric                     | Phase 1 | Phase 2 | Phase 3   |
|----------------------------|---------|---------|-----------|
| Active farmers             | 500     | 5,000   | 50,000    |
| Weekly retention rate      | > 40%   | > 50%   | > 60%     |
| Disease detection accuracy | > 70%   | > 80%   | > 85%     |
| AI resolution rate         | > 50%   | > 65%   | > 75%     |
| Languages supported        | 2       | 5       | 12        |
| Products in catalog        | 50      | 500     | 2,000     |
| Monthly GMV                | 5 Lakh  | 40 Lakh | 1.2 Crore |
| Net Promoter Score (NPS)   | > 20    | > 30    | > 40      |
| Average response time      | < 5 sec | < 4 sec | < 3 sec   |

## 12 Conclusion

### 12.1 Architecture Summary

The AI Kisan Assistant platform is built on a robust, scalable, and secure four-layer architecture designed for the unique challenges of Indian agricultural contexts.

Table 28: Complete architecture summary

| Dimension            | Specification                | Key Technologies                     |
|----------------------|------------------------------|--------------------------------------|
| Architecture Pattern | 4-Layer Layered Architecture | Channel-Gateway-Service-Data         |
| AI Models            | 3 Production Models          | CropDoc-Net, PestWatch, FarmCredit   |
| Model Accuracy       | 86.7% (CropDoc-Net)          | EfficientNet-B4 + ResNet-50 ensemble |
| Deployment           | Multi-region AWS             | Mumbai (primary) + Hyderabad (DR)    |
| Availability SLA     | 99.9%                        | 30-minute RTO, 15-minute RPO         |
| Security             | Defense-in-depth             | AES-256, TLS 1.3, SOC 2 plan         |
| Scalability          | Horizontal scaling           | Kubernetes HPA to 500K+ users        |
| Languages            | 12 Indian languages          | Whisper ASR + ElevenLabs TTS         |

## 12.2 Key Technology Decisions

Table 29: Technology decision matrix

| Component              | Selected Technology              | Rationale                                                      |
|------------------------|----------------------------------|----------------------------------------------------------------|
| API Gateway            | Kong (open-source)               | Plugin ecosystem, high performance (10K+ req/s), rate limiting |
| Event Bus              | Apache Kafka                     | Durability, replayability, high throughput (1M msg/s)          |
| Primary Database       | PostgreSQL 15 + TimescaleDB      | ACID compliance + time-series extensions (10x compression)     |
| Cache                  | Redis ElastiCache                | Sub-millisecond latency, rich data structures (JSON, streams)  |
| Vector Database        | Qdrant Cloud                     | Fast similarity search (<50ms for 1M vectors), filtering       |
| LLM Strategy (Phase 1) | GPT-4o via API                   | Best accuracy (90%+ on AgriQA), multi-modal                    |
| LLM Strategy (Phase 3) | Llama 3 70B (fine-tuned)         | Open source, data privacy, lower cost (0.008/query)            |
| Computer Vision        | EfficientNet-B4 + ResNet-50      | 86.7% ensemble accuracy, bias-variance trade-off               |
| Speech                 | Whisper (ASR) + ElevenLabs (TTS) | 98% Hindi accuracy, 12 languages, natural voice                |
| Orchestration          | AWS EKS (Kubernetes)             | Portability, auto-scaling (HPA), managed control plane         |
| CI/CD                  | GitHub Actions + ArgoCD          | GitOps workflow, automated rollback, image scanning            |
| Monitoring             | Prometheus + Grafana + Jaeger    | Three-pillar observability (metrics, logs, traces)             |
| Infrastructure         | Terraform (HashiCorp)            | Infrastructure as Code, state locking, module reuse            |

## 12.3 Immediate Next Steps

Table 30: Week 1-7 action plan

| Day | Action Item                                                        | Responsible |
|-----|--------------------------------------------------------------------|-------------|
| 1   | Provision AWS accounts, set up IAM roles with least privilege      | CTO         |
| 2   | Deploy EKS cluster in Mumbai region with 3 availability zones      | DevOps      |
| 3   | Configure Kong API gateway with SSL certificates and rate limiting | DevOps      |
| 4   | Set up RDS (PostgreSQL 15 + TimescaleDB) with Multi-AZ enabled     | Backend     |
| 5   | Deploy Kafka (MSK) with 12 partitions, 3 brokers, 7-day retention  | Backend     |
| 6   | Implement CI/CD pipeline (GitHub Actions + ArgoCD)                 | DevOps      |
| 7   | Deploy monitoring stack (Prometheus + Grafana + Loki + Jaeger)     | DevOps      |

## 12.4 Final Remarks

This architecture document provides a complete, production-ready blueprint for the AI Kisan Assistant platform. The design prioritizes:

- **Availability:** 99.9% SLA with multi-region DR (RTO 30 min, RPO 15 min)
- **Scalability:** Horizontal scaling of all stateless services based on real-time metrics
- **Security:** Defense-in-depth with encryption (AES-256/TLS 1.3), RBAC, and audit logging
- **Observability:** Three-pillar observability (metrics, logs, traces) for full system visibility
- **AI Quality:** Ensemble models with confidence thresholds (70%) and human-in-loop fallback

The platform is ready for immediate implementation beginning with **Phase 1 (Weeks 1-12)**.